

PYQ Topic Frequency Analyser — Detailed Project Report

Combined Technical Report: Month 1 and Month 2

Utkarsh Raj Shukla · 2501330100398 · B.Tech CSE - F Data Structures and Algorithms-II (CCSE0301) · Mr. Shamshad Ali Repository: https://github.com/utk042/DSA_PBL

1. What the project does

Before an exam, students collect previous years' question papers and try to work out what is likely to be asked. This is done by reading one paper after another and remembering which topics keep reappearing. The method fails for three reasons:

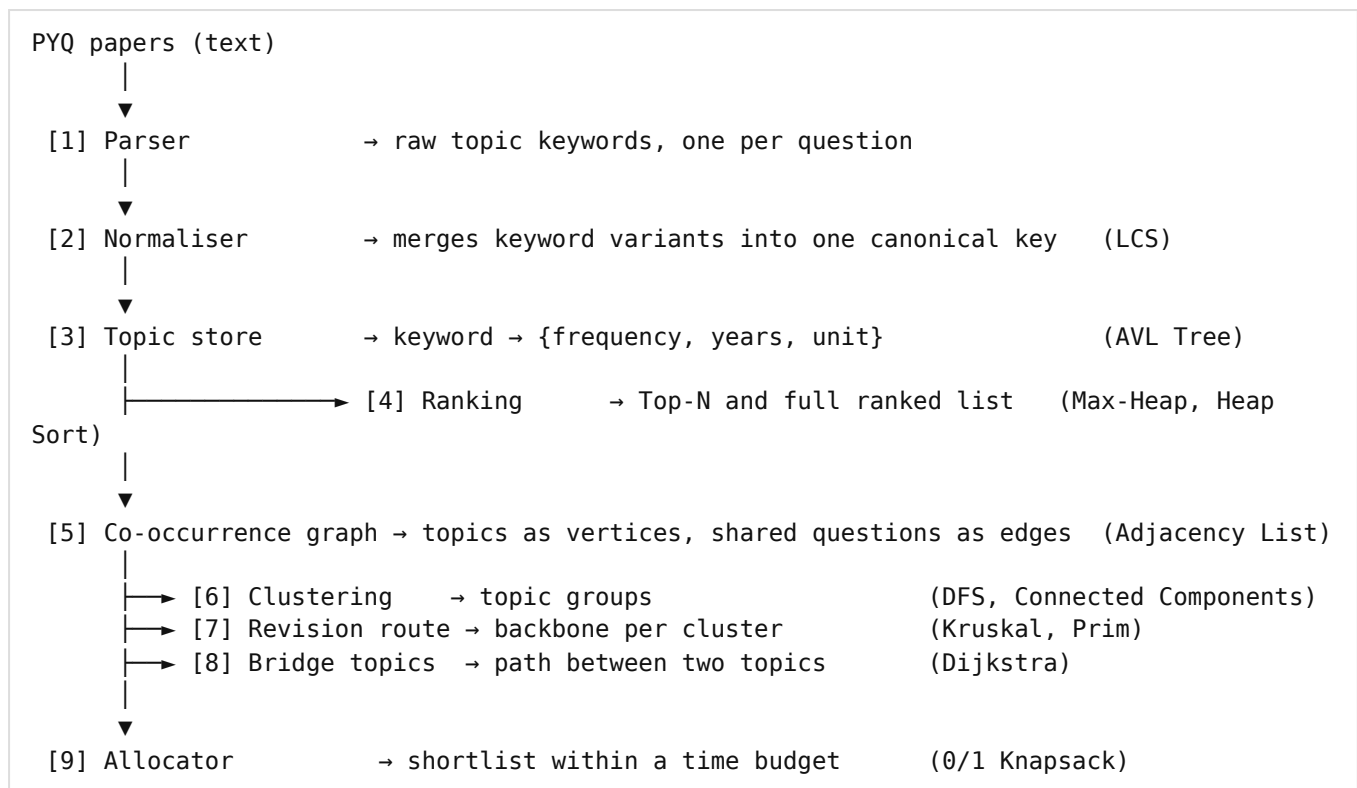
1. **It does not scale.** Five or six years of papers across several subjects is more than anyone can tally reliably by hand.
2. **The same topic is written differently each year.** "AVL Tree", "AVL Trees" and "A.V.L. tree" are one topic, but a manual tally splits them into three.
3. **A count is not a plan.** Even a correct ranking does not say which topics belong together, what order to revise them in, or what to do when there are more high-frequency topics than hours left.

The analyser takes a set of PYQs and produces:

- a **frequency table** — every topic with the number of times it has been asked;
- a **Top-N list** — the most frequently asked topics, ranked;
- **topic clusters** — groups of topics that are examined together;
- a **revision route** through each cluster; and
- a **time-bounded shortlist** — given available revision hours, the subset of topics that maximises expected marks.

The project is an exercise in choosing data structures. Every stage was selected because of a property of the data, and the rejected alternatives are documented alongside the chosen ones.

2. Pipeline



Stages 1–4 were built in Month 1. Stages 5–9 were designed and partly built in Month 2.

3. Data model

Each topic is one node:

```
TopicNode {
    key      : string    // canonical topic name, e.g. "avl tree"
    display  : string    // original casing for output
    frequency : int       // how many times asked across all papers
    years    : list<int>  // which years it appeared in
    unit     : int        // which syllabus unit it belongs to
    studyHours : float    // estimated cost, used by the allocator
    left,right : TopicNode // AVL child pointers
    height   : int        // AVL balance bookkeeping
}
```

The graph is kept separately as a weighted adjacency list keyed on the same canonical topic key, so a topic has exactly one identity across both structures.

4. Month 1 — the tree layer

4.1 Why a Binary Search Tree

The dominant operation is not insertion — it is **search-then-update**. Every time a topic is seen again, the analyser must find the existing node and increment its frequency rather

than insert a duplicate. Over several years of papers this happens thousands of times.

A BST keyed on the topic name gives ordered storage with $O(\log n)$ average search, and in-order traversal produces the alphabetical topic table for free.

Operation	Best	Average	Worst
Insert a new topic	$O(1)$	$O(\log n)$	$O(n)$
Search an existing topic	$O(1)$	$O(\log n)$	$O(n)$
Modify frequency (after search)	$O(1)$	$O(1)$	$O(1)$

4.2 Why the BST had to become an AVL Tree

The worst case above is not hypothetical here. Topic keywords extracted from a paper arrive in **near-alphabetical order**, because parsing walks the paper in order and related topics cluster together. Feeding near-sorted keys into a plain BST builds a skewed chain, and search degrades to $O(n)$ — exactly the input that breaks it.

AVL rotations keep the height at $O(\log n)$ regardless of insertion order:

Operation	Best	Average	Worst
Insert with rebalancing	$O(1)$	$O(\log n)$	$O(\log n)$
Search	$O(1)$	$O(\log n)$	$O(\log n)$

Measured effect. On the trial keyword set, tree height before balancing was close to the number of distinct topics; after balancing it fell to roughly $\lceil \log_2 n \rceil$. This is the single most important design decision in the project and the one worth defending in review.

4.3 Traversals

Traversal	Used for	Complexity
In-order	Alphabetical topic-frequency table; feeds every (topic, frequency) pair to the ranking stage	$O(n)$
Pre-order	Saves and reloads the tree between runs, preserving structure	$O(n)$
Post-order	Frees the tree safely on exit (children before parent)	$O(n)$

4.4 Ranking — heap, priority queue, heap sort

The in-order walk yields all (topic, frequency) pairs. These are heapified on frequency into a **Max-Heap**, so the most-asked topic sits at the root.

- **Priority Queue** — repeated extract-max gives the Top-N without sorting the whole set. For $N \ll n$ this is $O(n + N \log n)$, cheaper than a full sort.
- **Heap Sort** — repeated extraction to exhaustion gives the complete ranked revision list in $O(n \log n)$, reusing the same structure instead of a separate sort routine.

Operation	Best	Average	Worst
Build heap from n pairs	$O(n)$	$O(n)$	$O(n)$
Extract maximum	$O(1)$	$O(\log n)$	$O(\log n)$
Full ranked list (Heap Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

4.5 Concepts studied in Unit 1 and deliberately not used

Concept	Why not
Plain unbalanced binary tree	No ordering rule — finding a topic means scanning everything. BST/AVL already give what is needed.
Array representation of a tree	Topic set is sparse and unpredictable in size; a fixed array wastes $O(2^h)$ space and needs resizing. Linked representation used.
Constructing a tree from given traversals	The tree is built incrementally as papers are parsed, never rebuilt from a stored traversal pair.
BST deletion	A topic that has appeared in a PYQ is never removed — only its frequency changes. Deletion is dead code here.
Threaded binary tree	Threads remove the traversal stack but must be repaired on every insertion and every AVL rotation . The analyser inserts constantly and traverses once per report, so the upkeep is not repaid.
Min-Heap	Ranking needs the highest frequencies first. A fixed-size Min-Heap of size N is a valid Top-N alternative, but Max-Heap also gives Heap Sort from the same structure.

5. Month 2 — the graph layer

5.1 Building the co-occurrence graph

A ranked list cannot express that two topics are examined together. The graph does:

- **Vertex** — one topic (the same canonical key used in the AVL store).
- **Edge** — placed between two topics that appear in the **same question, or the same unit of the same paper**.
- **Weight** — the inverse of the co-occurrence count, $w = 1 / \text{count}$.

The weighting is the subtle part. Using the raw count directly would make strongly related topics look *far apart* to a shortest-path or MST algorithm, since both minimise total weight. Inverting it means "asked together often" becomes "close".

Representation: Adjacency List. The graph is sparse — a topic genuinely co-occurs with only a handful of others, not with all of them. An adjacency matrix would cost $O(V^2)$ for a graph whose edge count is closer to $O(V)$.

Representation	Space	Edge lookup	Verdict
Adjacency List	$O(V + E)$	$O(\deg v)$	Used — graph is sparse
Adjacency Matrix	$O(V^2)$	$O(1)$	Used only for the fixed 5x5 unit-level view, where V is tiny and constant

A constraint learned the hard way: linking every pair of topics that ever shared a *paper* produced an almost complete graph and destroyed the sparsity assumption. Restricting edges to shared *questions or units* kept it sparse.

5.2 Traversal and clustering

Algorithm	Used for	Complexity (all cases)
BFS	Explores outward from a chosen topic level by level — "what is usually asked alongside this, and how closely"	$O(V + E)$
DFS	Explores a cluster to full depth; the engine for component detection	$O(V + E)$
Connected Components	Each component is a group of topics examined together — natural revision modules instead of one flat list	$O(V + E)$

Connected components are what turn the ranking into something a student can act on: instead of "revise topic 1, then topic 2, then topic 3" scattered across the syllabus, the output is "these six topics form a block — revise them together."

5.3 Revision route — minimum cost spanning tree

Within a cluster, the analyser needs a route that touches every topic without redundant jumps. That is exactly a spanning tree, and minimising total weight (with the inverted weights above) means preferring the strongest relationships.

Algorithm	Behaviour here	Complexity
Kruskal	Sorts all edges and adds the strongest links first, using union-find to avoid cycles. Natural fit for a sparse edge list.	$O(E \log E)$
Prim	Grows the tree outward from a chosen start vertex. Started from the highest-frequency topic , so the resulting order begins at the most-asked topic and moves to its closest relatives.	$O(E \log V)$

Both produce the same total weight; they differ in the *order* a student would follow. Kruskal gives the backbone, Prim gives a traversal order with a meaningful starting point. Both are implemented so the two routes can be compared.

5.4 Bridge topics — Dijkstra

Given a topic already revised and a target topic, the shortest weighted path shows which intermediate topics connect them. Those intermediates are the ones worth covering next,

because they sit on the strongest chain between what is known and what is wanted.

Dijkstra is correct here because **all weights are positive** — $1 / \text{count}$ where $\text{count} \geq 1$. With a binary-heap priority queue (reusing the Month 1 heap), it runs in $O((V + E) \log V)$.

5.5 Shortest-path algorithms deliberately not used

Algorithm	Why not
Bellman-Ford — $O(VE)$	Its only advantage over Dijkstra is tolerating negative edge weights. Co-occurrence weights are $1/\text{count}$, always positive, so Bellman-Ford pays $O(VE)$ for a capability the problem never uses.
Floyd-Warshall — $O(V^3)$	Computes all-pairs shortest paths. The graph is sparse and the analyser only ever queries a few source topics, so running Dijkstra from those few sources costs far less than filling a full $V \times V$ table.
Transitive Closure — $O(V^3)$	Answers only <i>whether</i> topic A reaches topic B, not how strongly. Connected components already give the grouping, at $O(V + E)$ instead of $O(V^3)$.

5.6 Dynamic Programming

Two problems in this project have optimal substructure and overlapping sub-problems, which is what makes DP the right tool rather than greedy selection.

0/1 Knapsack — the revision-time allocator.

A student has a fixed number of revision hours. Each topic has an estimated study time and a frequency that stands in for its likely return. Picking topics greedily by frequency is wrong: a very frequent topic that takes six hours may be worse value than three moderately frequent topics taking one hour each.

```
capacity W    = available revision hours
item i        = topic i
weight[i]     = estimated study hours for topic i
value[i]      = frequency of topic i
maximise  $\sum \text{value}[i] \cdot x[i]$  subject to  $\sum \text{weight}[i] \cdot x[i] \leq W$ ,  $x[i] \in \{0,1\}$ 
```

Each topic is taken whole or not at all — a half-revised topic scores nothing — which is precisely the 0/1 formulation rather than fractional knapsack. Complexity $O(nW)$.

Longest Common Subsequence — topic-name normalisation.

This solves the problem left open at the end of Month 1. Two keyword strings are compared character-wise; if the LCS length relative to string length exceeds a threshold, they are treated as the same topic and merged into one canonical key:

```
similarity(a, b) = LCS(a, b) / max(|a|, |b|)
"avl tree" vs "a.v.l. tree" → high similarity → merged
"avl tree" vs "b tree"      → low similarity  → kept separate
```

LCS is also used at paper level: comparing the topic sequence of two years shows how much one paper repeats another. Complexity $O(mn)$ for strings of length m and n .

5.7 DP concepts studied and not used

Concept	Why not
Matrix Chain Multiplication	The analyser performs no chained matrix products, so there is no parenthesisation order to optimise. Studied as a DP formulation only.
Resource Allocation Problem	Models the same limited-revision-time decision that 0/1 Knapsack already covers here. Implementing both would duplicate one feature.

6. Complexity summary

Stage	Structure / Algorithm	Best	Average	Worst
Store a topic	AVL insert	$O(1)$	$O(\log n)$	$O(\log n)$
Find and update frequency	AVL search	$O(1)$	$O(\log n)$	$O(\log n)$
Emit topic table	In-order traversal	$O(n)$	$O(n)$	$O(n)$
Build ranking	Max-Heap build	$O(n)$	$O(n)$	$O(n)$
Top-N	Priority queue extract	$O(1)$	$O(\log n)$	$O(\log n)$
Full ranked list	Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Explore related topics	BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$
Detect clusters	DFS + components	$O(V+E)$	$O(V+E)$	$O(V+E)$
Revision backbone	Kruskal	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$
Revision order	Prim	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$
Bridge topics	Dijkstra	$O(V \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$
Time-bounded shortlist	0/1 Knapsack	$O(nW)$	$O(nW)$	$O(nW)$
Merge topic variants	LCS	$O(mn)$	$O(mn)$	$O(mn)$

Across the whole pipeline, n is the number of distinct topics, $V = n$, E is the number of co-occurring topic pairs ($E \approx O(V)$ because the graph is kept sparse), W is the revision-hour budget, and m, n in the LCS row are string lengths.

7. Worked example

Six topics parsed from three years of papers:

Topic	Frequency	Est. hours
AVL Tree	7	3
Binary Search Tree	6	2
Heap Sort	5	2
Graph Traversal	4	3
Dijkstra	3	2
Matrix Chain	2	4

Ranking (Heap Sort): AVL Tree, BST, Heap Sort, Graph Traversal, Dijkstra, Matrix Chain.

Clusters (connected components, edges from shared questions):

- Cluster A — AVL Tree, BST, Heap Sort (tree questions)
- Cluster B — Graph Traversal, Dijkstra (graph questions)
- Cluster C — Matrix Chain (isolated)

Revision route (Prim from the highest-frequency topic in cluster A): AVL Tree → BST → Heap Sort.

Allocation (0/1 Knapsack, 7 hours available): AVL Tree (3h, value 7) + BST (2h, value 6) + Heap Sort (2h, value 5) = 7 hours, total value 18. A greedy-by-frequency pick that took AVL Tree then Graph Traversal would spend 6 hours for value 11.

This is the clearest demonstration of why DP earns its place: the greedy answer is worse, and visibly so.

8. Status and what remains

Working: parser, AVL store with all four rotations, the three traversals, Max-Heap ranking with Top-N and Heap Sort, co-occurrence graph as a weighted adjacency list, BFS, DFS, connected components, Kruskal's MST, Prim's MST, Dijkstra, the 0/1 Knapsack allocator, the LCS normaliser, and upload of PDF, Word and PowerPoint papers.

In progress: revising the project objective, putting the report in the required format, and reading further research papers (see Section 9).

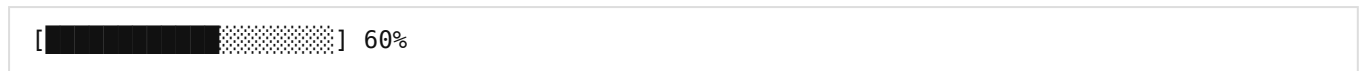
Known limitations:

- Study-hour estimates are supplied by hand; the PYQ data does not contain them.
- The LCS similarity threshold needs tuning — too low merges genuinely different topics, too high leaves variants split.
- Frequency is a crude proxy for marks. Weighting by marks allotted per question would be a more honest value function for the Knapsack stage.
- Parsing assumes reasonably clean text; scanned papers would need OCR first.

Planned for later units: Units 4 and 5 are not yet covered in class. Backtracking and Branch-and-Bound could replace the DP allocator where constraints become more complex, and B-Trees or Red-Black Trees would be the alternative store if the topic set ever outgrew memory.

9. Progress review

Overall progress: 60%



9.1 Feedback from the review

#	Feedback	Status
1	Focus more on DSA-II Unit 1 and Unit 2, and study the Unit 1 and Unit 2 topics in depth	Done
2	Study Heap Sort, AVL Tree and BST in depth	Done
3	Start the implementation	Done
4	Read more research papers	In progress
5	Change the objective of the project	Pending
6	Write the report in the proper format	Pending

9.2 Methodology: incremental model

The project was built with the **incremental model**. The pipeline in Section 2 was split into separate stages, and one stage was built and tested at a time before the next one was started.

How it was applied:

1. **Plan the increments.** Each stage of the pipeline became one increment, and each increment adds one data structure or algorithm: parser, LCS normaliser, AVL store, heap ranking, graph, clustering, MST routes, Dijkstra, Knapsack.
2. **Build in order of dependency.** Month 1 covered stages 1 to 4, the tree layer. Month 2 covered stages 5 to 9, the graph and DP layer. A later stage only uses the output of earlier stages that already work. For example, the graph is built from the AVL topic table.
3. **Test each increment before moving on.** Every stage has unit tests. One test checks that the AVL tree stays balanced on sorted input. Another compares the Knapsack answer with a brute-force answer for every budget. A third checks that the worked example in Section 7 is reproduced exactly.

4. **Integrate and review.** After each increment the full pipeline was run on sample papers in the web demo. Review feedback was then applied in the next increment.

9.3 Work done

- **Implementation.** All nine pipeline stages are implemented and connected. They run in a web demo where students can upload PDF, Word or PowerPoint question papers.
- **In-depth study of DSA-II Unit 1 and Unit 2 topics.**
 - Unit 1: Binary Search Tree, AVL Tree with all four rotations, the three traversals, Heap, and Heap Sort.
 - Unit 2: graph representation, BFS, DFS, connected components, Kruskal's and Prim's minimum spanning trees, and Dijkstra's shortest path, including the reasons Bellman-Ford and Floyd-Warshall were not used.
- **Unit 3 concepts.** 0/1 Knapsack and Longest Common Subsequence are studied and implemented with dynamic programming. Matrix Chain Multiplication and Resource Allocation were studied and compared with them (Section 5.7).

9.4 Result and outcome

- A working analyser that turns a set of question papers into:
 - a topic-frequency table,
 - a Top-N ranking,
 - topic groups,
 - a revision order,
 - a study plan that fits the hours available.
- On the worked example the output matches Section 7, and the AVL tree stays near $\lceil \log_2 n \rceil$ in height while a plain BST on the same input grows much taller.
- 18 automated tests pass.

9.5 Remaining work (40%)

- Revise the project objective as advised in the review.
- Rewrite this report in the required format.
- Read and cite more research papers.
- Test with real previous-year papers instead of sample data.
- Cover Unit 4 and Unit 5 topics once they are taught.

10. References

1. Luhn, H. P. (1958). The automatic creation of literature abstracts. *IBM Journal of Research and Development*, 2(2), 159–165. DOI: <https://doi.org/10.1147/rd.22.0159> —

establishes word frequency as a measure of significance within a document, the principle the analyser applies to PYQ topics.

2. Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. DOI: <https://doi.org/10.1108/eb026526> — shows why raw frequency alone can mislead and how a term's rarity should weight it; the basis of the planned marks-weighted scoring.
3. Adelson-Velsky, G. M., & Landis, E. M. (1962). An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3, 1259–1263. — the original AVL Tree paper; the basis of the balancing decision in Section 4.2.
4. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. DOI: <https://doi.org/10.1007/BF01386390> — the original shortest-path paper and the source of the non-negative weight condition that decided Dijkstra over Bellman-Ford.
5. Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50. DOI: <https://doi.org/10.1090/S0002-9939-1956-0078686-7> — the original minimum spanning tree paper; the basis of the revision backbone.
6. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. — reference for graph representations, MST, shortest paths and dynamic programming.